hhu

hhu
Heinrich Heine
Universität Düsseldorf

Isolation and Protection in Operating Systems

4. Paging in x86-64

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

Current State

Heinrich Heine
Universität Düsseldorf

- We use segmentation to differentiate between kernel and user code
  - User mode threads cannot execute privileged CPU instruction
  - We also blocked accesses to I/O-ports for user mode threads via the TSS

- We now implement paging to restrict memory accesses
  - This enables us to isolate user mode processes...
  - ...and protect the kernel's memory area from being accessed by user mode code

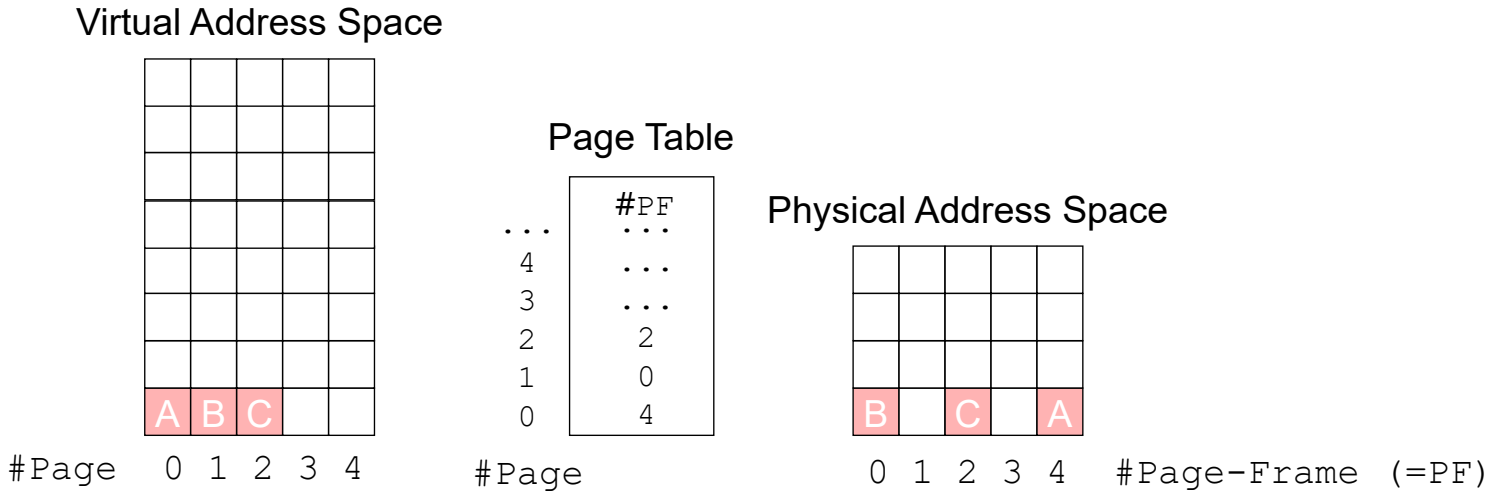
Paging

Heinrich Heine
Universität Düsseldorf

- Virtual address space is divided into equal sized *pages*
- Virtual address space is divided into equal sized *page frames*
- Usually 4 KiB pages and thus 4 KiB page frames are used
  - One page frame stores exactly one page
  - Special case: *Huge Pages*, e.g. 2 MiB or 1 GiB
- Memory protection can be specified with a 4 KiB granularity
  - Read only
  - Execute
  - User mode access allowed/forbidden

Virtual vs. Physical Addresses

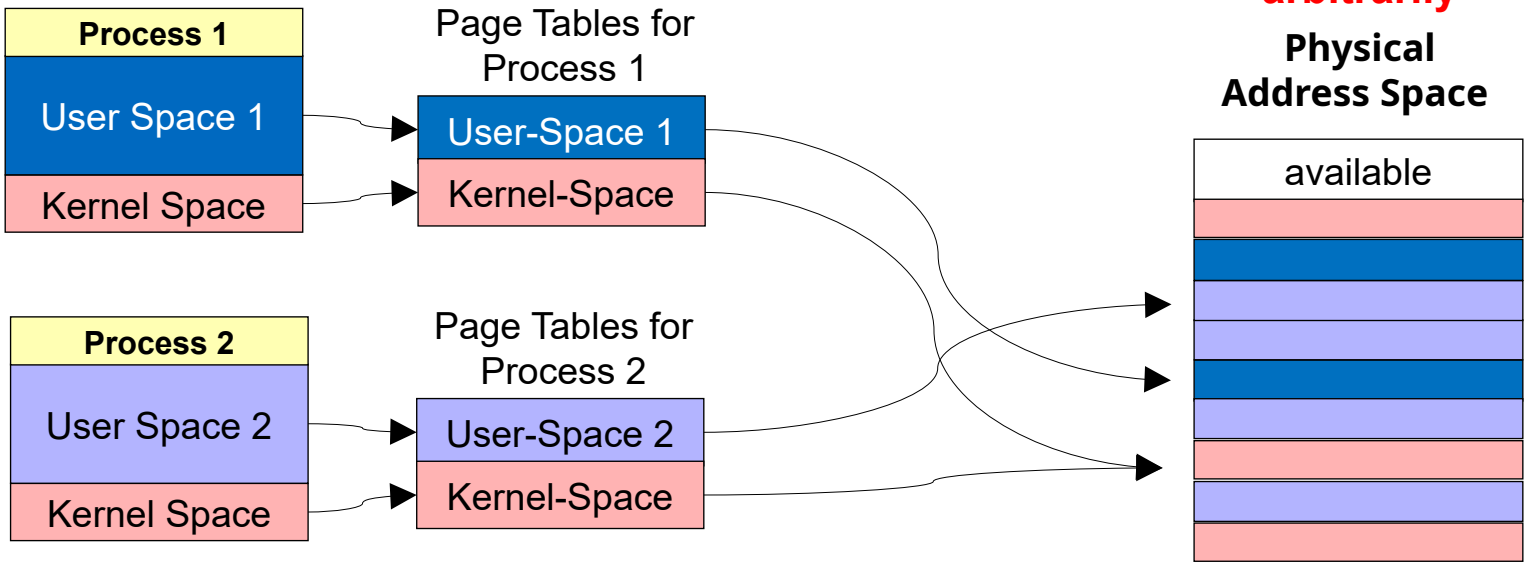
- Virtual/Logical addressesare differentiated from physical addresses
  - Virtual addresses use up to 57 bit, depending on the CPU
  - Addressable physical memory depends on the installed DRAM
  - A block of virtual memory can be mapped to arbitrary page frames



Process Isolation

- Each process has its own page tables describing its virtual address space
- We map the kernel memory into every address space

Page frames can be mapped arbitrarily



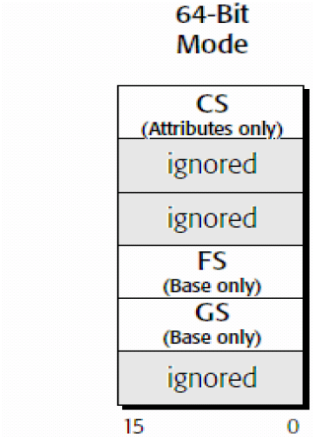
Process Isolation

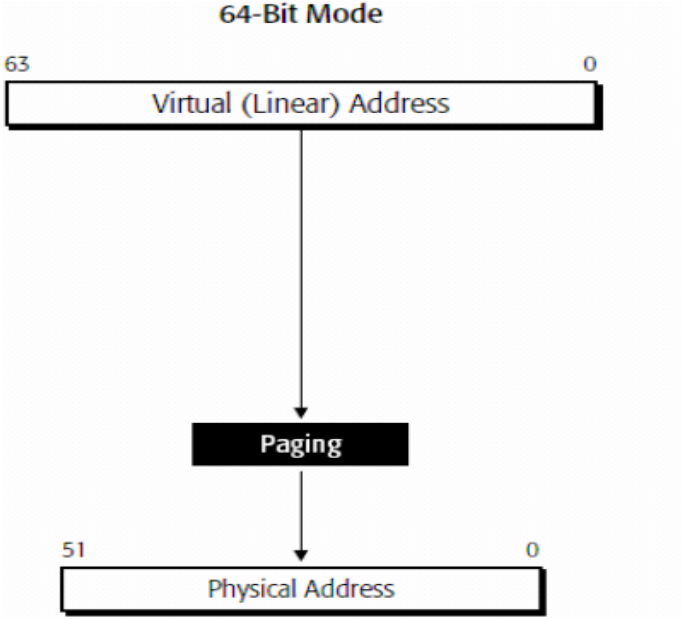
Heinrich Heine
Universität Düsseldorf

- Each thread runs in a process address space and can only access virtual addresses on pages that are allowed by the corresponding page table entries
- This is controlled by bits in the page table entries (see next slides)
- If a page is not *present*, it is either swapped out or unused
  - Accessing it causes a *Page Fault*
- The page tables themselves must be protected from user space accesses (just like the kernel)
  - This is controlled by a specific bit in the page table entries (see next slides)

Long Mode: Segmentation

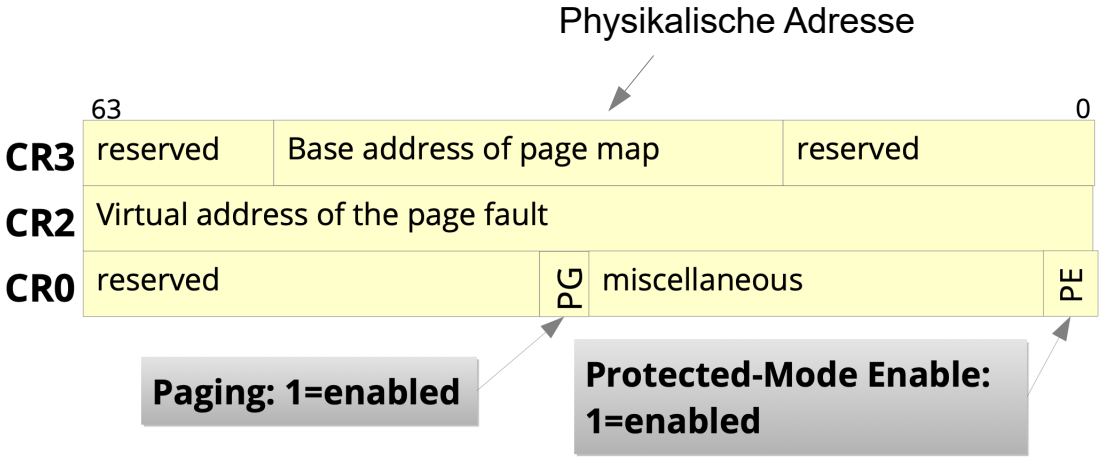
- 16-bit selectors
- Start addresses are ignored (except FS and GS)
  - FS: For thread-local storage (used by thread libraries)
  - GS: Linux uses it to retrieve the stack address for fast system calls
- No limit checks
  - Only the ring number in CS remains relevant





Long Mode: Paging

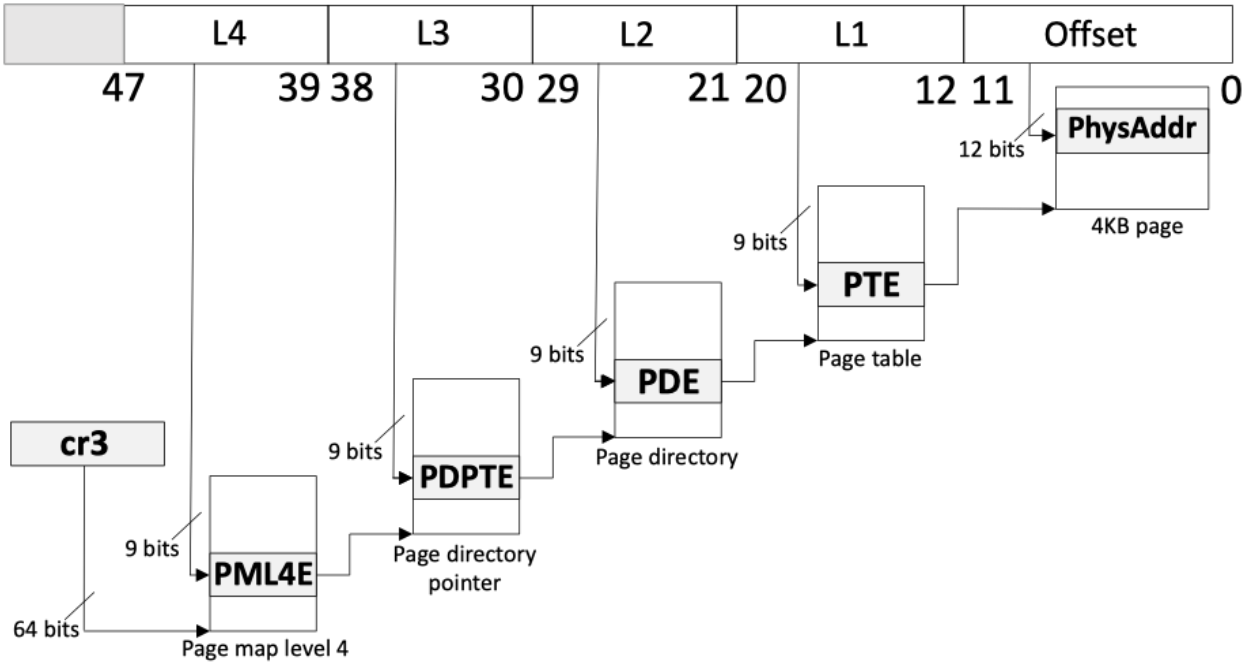
- Paging is mandatory in Long Mode
- The basic paging configuration is handled by the CRx **C**ontrol **R**egisters



Long Mode: Page Tables

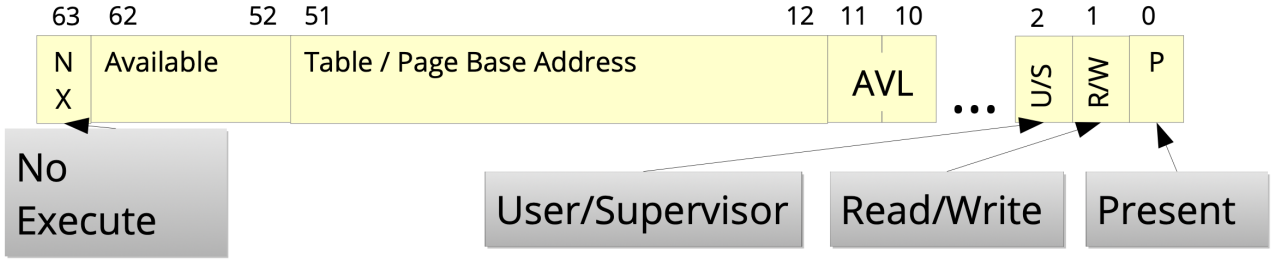
Heinrich Heine
Universität Düsseldorf

- Four level Paging
  - Modern CPUs may support five levels
- Index: 9 Bit
  - 4 KiB per table
  - 8 byte per entry
  - 512 entries per table



Page Table Entry

- Configuration is possible on all level of the page table hierarchy
- Protection bits:
  - R/W = Read/Write → Protection agains write accesses
  - NX = NoeXecute → Prohibit code execution in memory that only contains data
  - U/S = User/Supervisor → Access allowed for everyone or only in ring 0



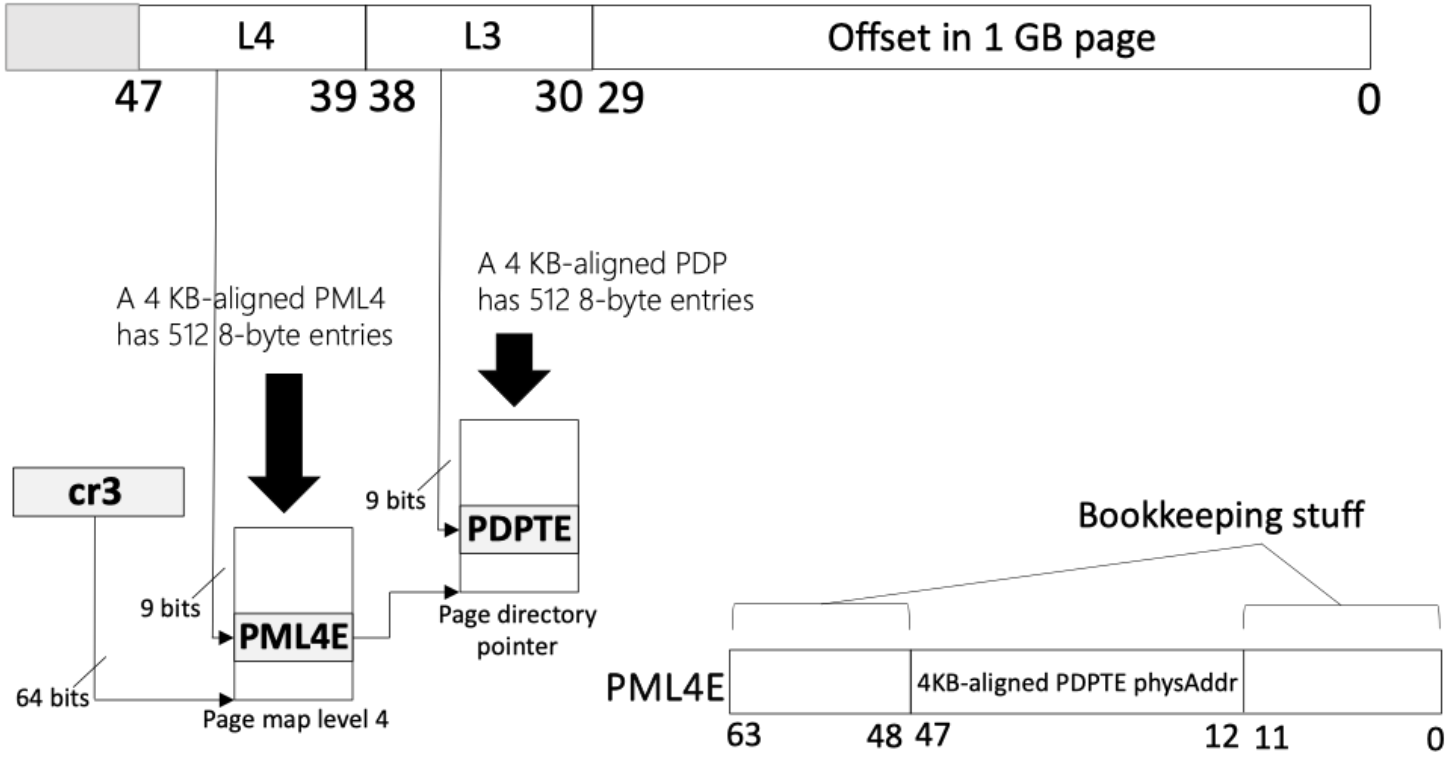
  - Available = AVL = Bits that are freely usable by the operating system

Bemerkungen

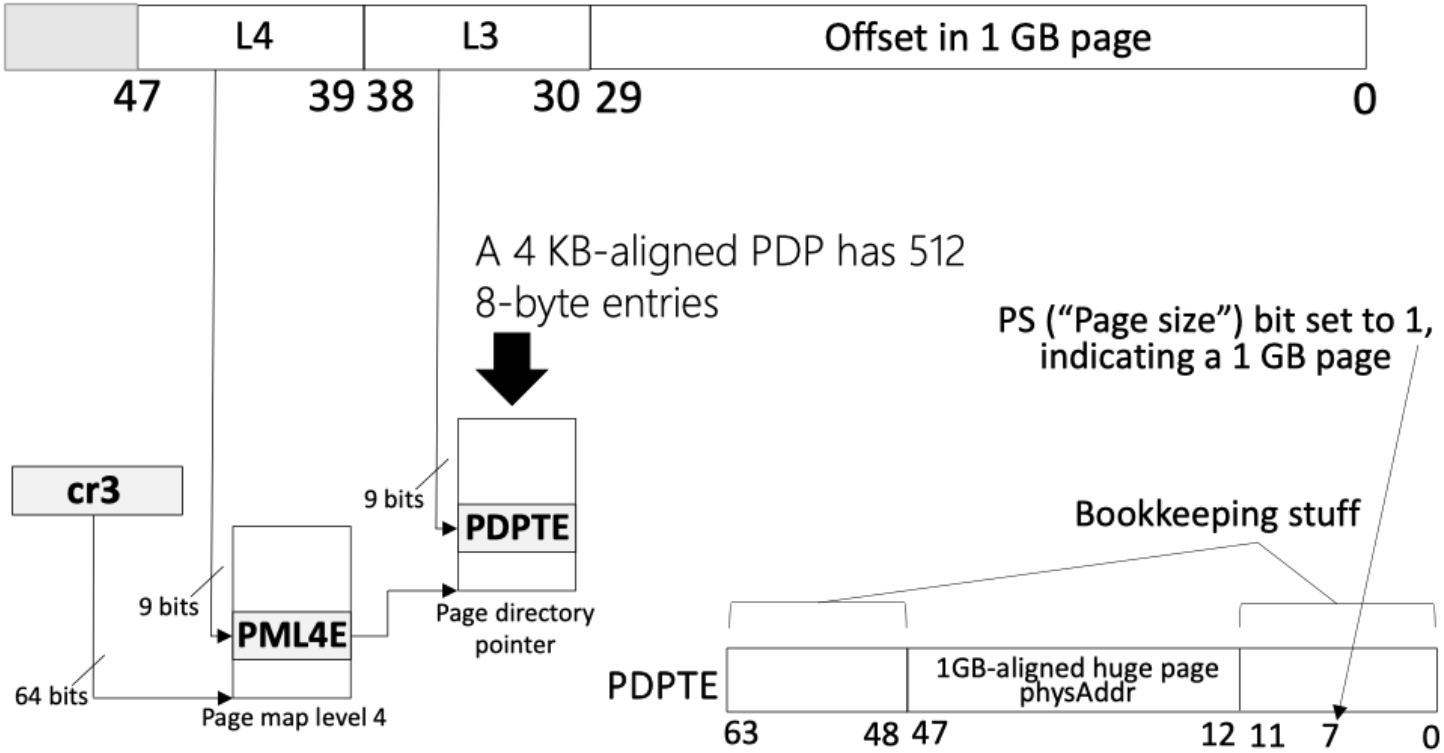
Heinrich Heine
Universität Düsseldorf

- Warum gibt es so viele Stufen?
  - I.d.R. wird der virtuelle Adressraum nur spärlich und verstreut genutzt
  - Allenfalls großer Heap bei Big-Data-Systemen
  - Aber Stacks und Code benötigen nicht Gigabytes
- Es werden nur notwendige Tabellen angelegt
  - Ein Eintrag in PML4E beschreibt 512 GB
  - Falls dieser Adressbereich komplett ungenutzt ist, so wird PML4E.present bit = 0 gesetzt
  - Und entsprechend werden nicht alle zugehörigen Tabellen PDP/PD/PT Tabellen in diesem Unterbaum angelegt

Example: 1 GiB Pages



Example: 1 GiB Pages



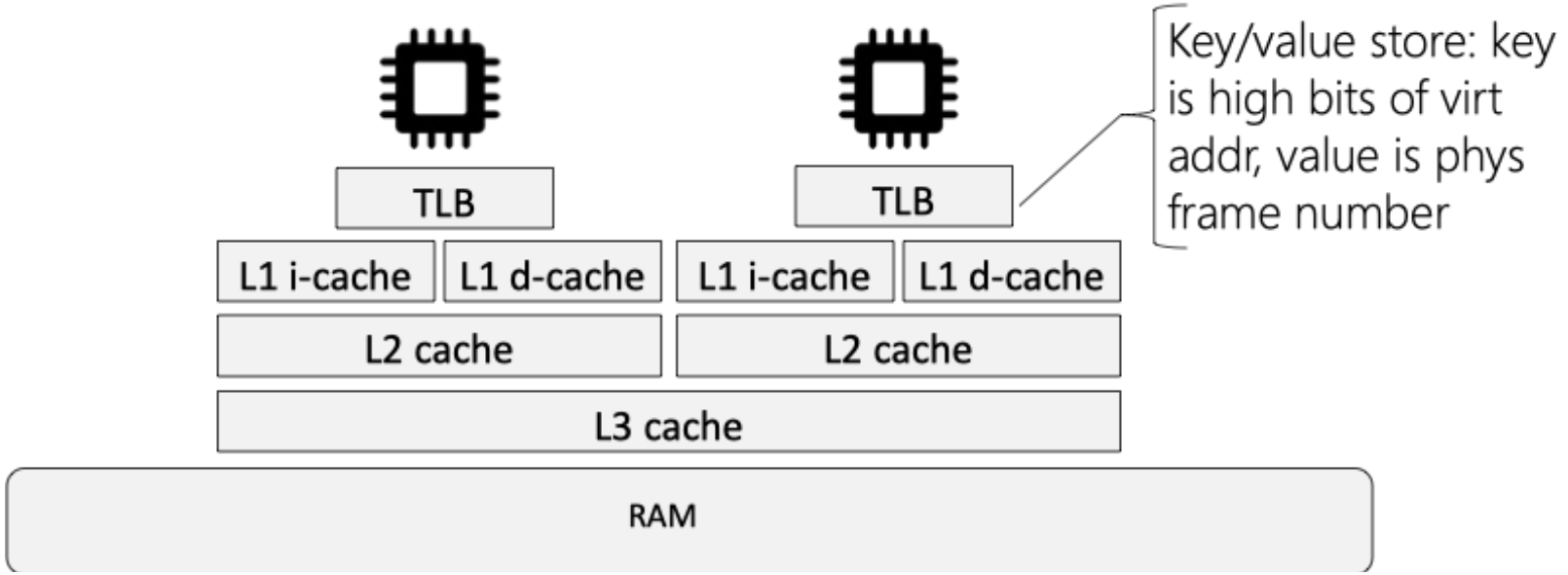
Long Mode: TLB

Heinrich Heine
Universität Düsseldorf

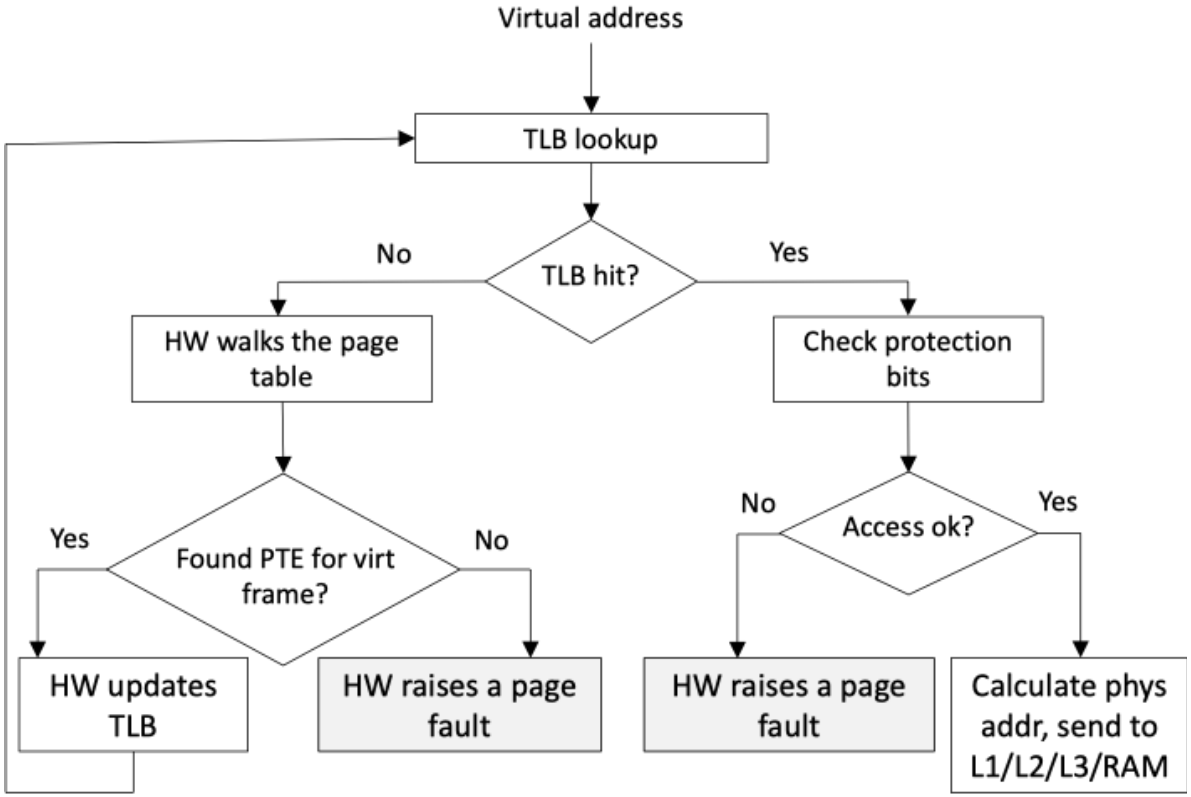
- Problem: Paging requires address translation by the *Memory Management Unit* (MMU) → One MMU per core
  - Address translation is slow because of the four/five levels of page tables
  - Long Mode requires paging

- Solution: Translation Lookaside Buffer (TLB) → One TLB per core:
  - Buffers translated addresses
  - Fully associative cache → Lookup in O(1)
    - Tag: consists of page table indices
    - Data: page frame address

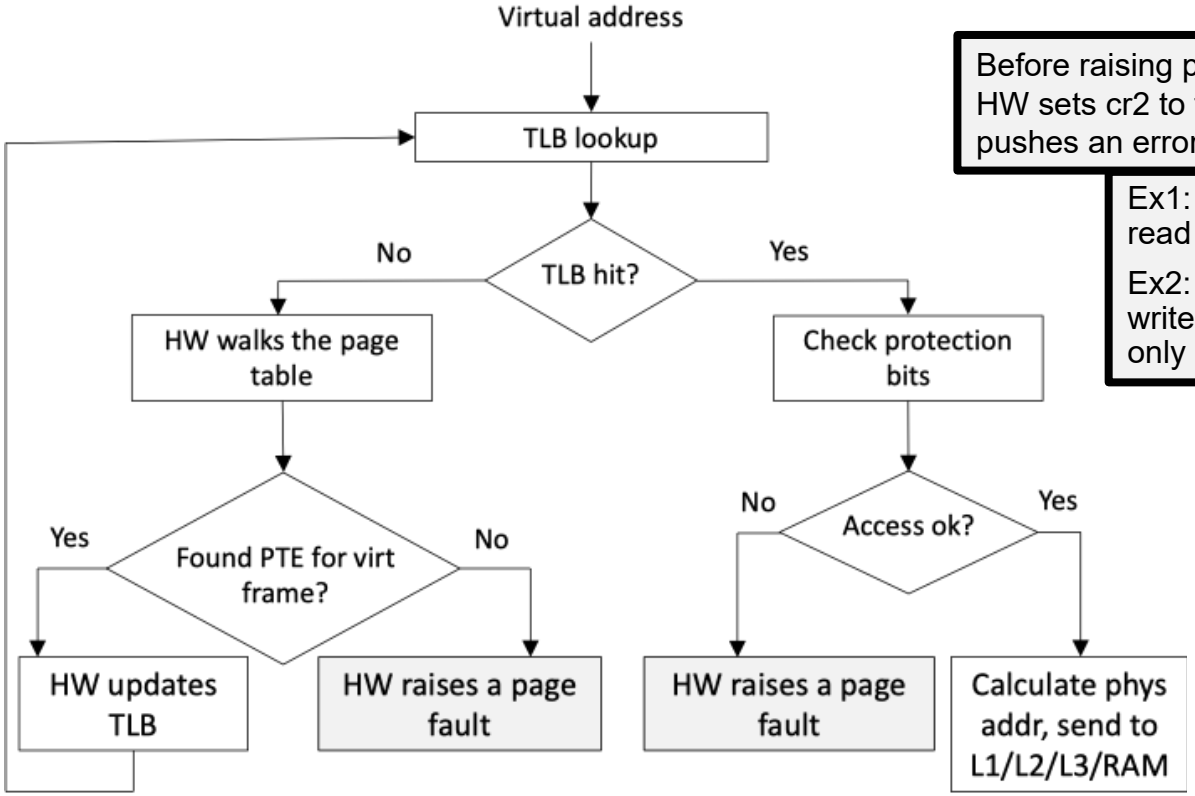
Long Mode: TLB



Long Mode: TLB



Long Mode: TLB



Before raising page fault exception, HW sets cr2 to faulting address, and pushes an error code onto stack

Ex1: User thread tried to read a non-present page

Ex2: User thread tried to write a present but read-only page

Long Mode: TLB

Heinrich Heine
Universität Düsseldorf

- TLB is small: ~32-100 entries (depending on the CPU)
- In usual application the TLB reaches a hit rate of up to 98%
  - Why? → Principle of locality
  - Arrays, Loops, etc.

- Remarks
  - Writing the CR3 register (address space switch) fully invalidates the TLB
  - Whenever protection/access bits in a page table are modified, the corresponding TLB entry must be invalidated manually (`INVLPG` instruction)
    - On multicore CPUs, this may be necessary on all cores!

To Be Continued...

Heinrich Heine
Universität Düsseldorf

- Kernel isolation after Meltdown
- Process Context Identifiers
- Memory protection keys